

Linguagem de Programação

Aula 05: Expressões, operadores de atribuição, funções integradas e entrada e saída de dados

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Na aula passada

- Aprendemos a utilzar o IDLE, tanto em sua interface de testes, o shell, quanto em seu editor de texto.
- Aprendemos sobre operadores aritméticos em Python
- Aprendemos sobre tipos de constantes: int, float, booleana e string
- Aprendemos a criar variáveis em Python e mais que isso, que a linguagem reconhece qual o tipo de variáveis, não sendo necessário ao longo de um programa (com exceção de entradas na tela) declara-lo.

Exemplo: nota_1 = 7.59 --> o Python reconhece que nota_1 é real. O operador = neste caso é de atribuição.

Na aula passada

- Aprendemos ainda a criar entradas simples na tela, para o usuário interagir com o programa:

variável = tipo(input(‘string: ’))

```
>>> n1 = float(input('Entre com a primeira nota:'))
Entre com a primeira nota:7.0
>>>
```

- Aprendemos ainda a criar saídas simples de um programa utilizando o comando print.

print(‘string’, variável a ser exibida)

```
>>> print('a nota n1 é:', n1)
a nota n1 é: 7.0
>>>
```

Mais um spoiler

Ainda vamos discutir formatos de saída, mas para fins de treinamento do beecrowd, neste curso, vamos utilizar o comando f-string que é muito versátil como veremos brevemente.

- Para imprimir um número real com um dado número de casas decimais, podemos proceder como no exemplo abaixo:

```
>>> import math as mt
>>> mt.pi
3.141592653589793
>>> print(f'pi = {mt.pi:.4f}')
pi = 3.1416
```

indica que a constante é float (real)

número de casas decimais

Expressões em Python

Expressões são combinações de operandos com zeros ou mais operadores fornecendo um valor.

- Lembrando que os operandos podem ser constantes ou variáveis.
- Quando uma instrução contém uma expressão, o Python avaliará a expressão.
- Em expressões inválidas, ocorrerão erros.

Expressões: Exemplos

```
>>> 3.1415   # expressão válida, resulta em 3.1415
>>> (2 + 5)  # expressão válida, resulta em 7
>>> 3.0 * 2  # expressão válida, resulta em 6.0
>>> 10 / 0   # expressão válida, porém resulta em erro de
               aritmética (erro de divisão por zero)
>>> +123     # expressão válida, resulta em 123
>>> 'oi'     # expressão válida, resulta em 'oi'
>>> 123+     # expressão inválida, falta um operando à direita
>>> 2 * 3)   # expressão inválida, parênteses desbalanceados
```

Operadores de atribuição

Já conhecemos o operador = no Python que é um operador de atribuição simples, ou seja, ele atribui um dado valor a uma variável:

Exemplos: minha_idade = 54,

x = 10 + 5, etc.

Importante lembrar que a expressão à direita do sinal de igual **será avaliada**, ou seja, no exemplo anterior, x não vai receber a expressão 10 + 5; ele vai receber o valor resultante da avaliação da expressão, ou seja, 15.

Operadores de atribuição

- Há operadores de **atribuição composta (aumentada)** e sua utilização tem a ver com a concisão das instruções.
- É possível também substituir instruções com atribuição composta por atribuições simples combinadas com outros operadores.

Alguns operadores de atribuição composta

Operador	Descrição	Exemplos
=	Atribui o resultado da expressão à variável.	x = expressão
+=	Atribui o resultado da adição à variável.	x += expressão x = x + (expressão)
-=	Atribui o resultado da subtração à variável.	x -= expressão x = x - (expressão)
*=	Atribui o resultado da multiplicação à variável.	x *= expressão x = x * (expressão)
/=	Atribui o quociente da divisão real à variável.	x /= expressão x = x / (expressão)
//=	Atribui o quociente da divisão inteira à variável.	x //= expressão x = x // (expressão)
%=	Atribui o resto da divisão inteira à variável.	x %= expressão x = x % (expressão)
**=	Atribui o resultado da exponenciação à variável.	x **= expressão x = x ** (expressão)

Alguns operadores de atribuição composta

Exemplo. Faça as seguintes operações no papel e depois veja no Shell do IDLE se você de fato entendeu.

```
>>> m = 2
>>> m **= 5  -> m = (m ** 5) = (2 ** 5) = 32
>>> m //= 4  -> m = (m // 4) = ( 32 // 4) = 8
>>> m += 1 ->  m = (m + 1) = ( 8 + 1) = 9
>>> m /= 2. -> m = (m / 2.) = 4.5
>>> m %= 2 -> m = (4.5 % 2) = 0.5
>>> m -= 0.2 -> m = (0.5 – 0.2) = 0.3
>>> m *= 5 -> m = ( m * 5) = (0.3 * 5) = 1.5
```

Alguns operadores de atribuição composta

Exercício 1. Verifique as seguintes operações no papel e depois veja no Shell do IDLE se você de fato entendeu.

```
>>>a = 10
>>>b = 10
>>>c = 10
>>> a *= 2 + 3 -> a = a * (2 + 3) primeiro o Python avalia o lado direito e só então faz a atribuição composta.
>>>b = b * 2 + 3 -> b = (b * 2) + 3
>>>c = c * (2+3)
```

Alguns operadores de atribuição composta

Exercício 2. Verifique as seguintes operações no papel e depois veja no Shell do IDLE se você de fato entendeu.

```
>>> x = 10
>>> y = 10
>>> z = 10
>>> x += 2 * 3
>>> y = y + 2 * 3
>>> z = z + (2 * 3)
```

$x = x + (2 * 3) = x + 6 = 10 + 6 = 16$

Precedência e associatividade de operadores

Precedência é igual à prioridade na realização de uma operação.

Logo, partes de uma expressão com operadores de **maior precedência** são avaliadas **antes** daquelas com operadores de menor precedência.

É possível modificar a precedência natural com a inclusão de parênteses, assim como na matemática.

Precedência e associatividade de operadores

Exemplo: 4 + 2*5 = 4 + 10 = 14 enquanto que (4+2) * 5 = 6* 5 = 30

Em expressões com parênteses aninhados, a precedência é do mais aninhado para o menos aninhado.

Exemplo: (4 + (2*5))/2 = (4 + 10)/2 = 14/2 = 7

E quando os operadores tem a mesma precedência, por exemplo, 8 / 4 * 2? Multiplicação e divisão tem a mesma precedência e o Python resolve esse problema com **a associatividade** que é uma propriedade que define como os operadores de mesma precedência devem ser agrupados.

Precedência e associatividade de operadores

Os operadores podem ser associativos à esquerda, direita ou não associativos.

- **à esquerda:** as operações são agrupadas da esquerda para a direita. Logo, o operando está associado ao operador à sua esquerda
- **à direita:** as operações são agrupadas da direita para a esquerda. Logo, o operando está associado ao operador à sua direita.
- **Não associativos:** as operações não podem ser encadeadas

Cada linguagem tem suas próprias regras sobre associatividade

Precedência e associatividade de operadores

Em cada linha da tabela, o primeiro operador tem maior precedência e o último menor

Operador	Descrição	Associatividade
**	Exponenciação	À direita
+, - (unários)	Identidade e negação (inversão de sinal)	À esquerda
*, /, //, %	Multiplicação, divisão real, divisão inteira e resto da divisão	
+, - (binários)	Adição e subtração	
=, +=, -=, *=, /= //=, %=, **=	Atribuição (simples e composta)	Não associativos

Precedência e associatividade de operadores

Finalmente, respondendo a pergunta sobre 8/ 4 * 2, neste caso ambos os operadores **são associativos à esquerda**, logo o resultado é 4. [=(8/4)*2]

Precedência e associatividade de operadores

Exemplo 1: Calcule o valor de s no Shell. Dados a = 5, b = 4, c = 9, d = 7, e = 1, f = 2 e s = 10. $s \mathrel{+}= a + b ** (c - d) / e * f$

1) O par de parênteses prioriza a expressão interna, dando a ela maior precedência, porém, como a operação contém variáveis, primeiro devemos substituí-las por seus respectivos valores. Após ser avaliado, o par de parênteses é descartado:
   `s += a + b ** (9 - 7) / e * f` ⇨ `s += a + b ** 2 / e * f`
2) Precedência: `s += a + 4 ** 2 / e * f` ⇨ `s += a + 16 / e * f`
3) Precedência e associativ.: `s += a + 16 / 1 * f` ⇨ `s += a + 16.0 * f`
4) Precedência: `s += a + 16.0 * 2` ⇨ `s += a + 32.0`
5) Precedência: `s += 5 + 32.0` ⇨ `s += 37.0`
6) Único operador: `s += 37.0`
   - ⇨ Atribuição composta, pode ser reescrita como: `s = s + (37.0)`
   - ⇨ `s = 10 + (37.0)` ⇨ `s = 10 + 37.0` ⇨ `s = 47.0`

Precedência e associatividade de operadores

Exemplo 2: Avalie considerando a precedência e ou associatividade a expressão t = 1 + 2 - 3 + 4 - 5

1) Associatividade: `t = 1 + 2 - 3 + 4 - 5` ⇨ `t = 3 - 3 + 4 - 5`
2) Associatividade: `t = 3 - 3 + 4 - 5` ⇨ `t = 0 + 4 - 5`
3) Associatividade: `t = 0 + 4 - 5` ⇨ `t = 4 - 5`
4) Associatividade: `t = 4 - 5` ⇨ `t = -1`
5) Único operador: `t = -1`

Precedência e associatividade de operadores

Apostila capítulo 03

Exercício: Avalie considerando a precedência e ou, associatividade, a expressão u = 2 ** 1 ** 3

Exercício: Avalie v = ((2 ** 1) ** 3)

Exercício: Avalie w = (5*8) + ((9 // 7) % 2)

w = (5 * 8) + (1 % 2)

w = 40 + (1 % 2)

w = 40 + 1

w = 41

Precedência e associatividade de operadores

Atenção: Havendo dois pares de parênteses independentes, **o mais à esquerda é avaliado primeiro**.

- Os operadores de atribuição não são associativos em Python e, portanto, seu encadeamento pode gerar uma interpretação especial ou erro.

Teste no shell:

```
>>> a = b = c = ( d = 10)
>>> Erro (não há o que atribuir à c; atribuição não resulta em valor)
 >>> x = 4
>>> y = 5
>>> x += y += 1
>>> Erro: (operadores de atribuição composta não são associativos)
```

Funções

Funções: Uma função, que tem um nome, é uma sequência de instruções que realiza uma tarefa específica.

Tipos de funções:

- **Integradas:** disponibilizadas na própria linguagem e prontas para uso
- **Importadas:** criadas por outros programadores podendo ser incluídas no ambiente de programação para serem usadas
- **Definidas:** criadas pelo próprio programador para serem utilizadas no código fonte e/ou distribuídas para outros códigos.

Funções Integradas

Para chamarmos uma função integrada utilizamos seu *nome* seguido de um par de parênteses

Exemplo: print( ),

abs( ) → funções com um argumento

pow(,) → função com dois argumentos (base e o expoente)

Além do nome e dos parênteses, as funções precisam dos argumentos de entrada para gerarem uma saída que é o valor retornado (ou valor devolvido) pela função.

https://docs.python.org/pt-br/3/library/functions.html

Função print

Um função integrada que será muito utilizada é a função **print** que faz exatamente o o que seu nome diz. A função **print** possui uma quantidade indeterminada de argumentos e não foi projetada para retornar valor.

Alguns exemplos:

```
>>> print('Hello World!') Hello World! (1 argumento)
>>> print('2 + 2 =', 2+2) (2 argumentos)
2 + 2 = 4
>>> idade = 52
>>> print('Tenho', idade, 'anos’)
Tenho 52 anos
(3 argumentos)
```

Função print

- Por padrão, print exibe os valores de seus argumentos e uma quebra de linha '\n ' fazendo que com que o próximo print exiba seus dados na lingua seguinte.

```
print('boa ')
print('noite ')
print('gente ')
```

- Para que o último print não quebre a linha acrescentamos como
- último argumento da função um end = " "

```
# Exemplo sem quebra de linha
print("Olá", end=" ")
print("mundo!")

# Resultado: Olá mundo! (tudo na mesma linha)
```

Função input

Função input( ):

- o programa aguarda uma entrada antes de prosseguir

- converte o valor lido para **d**

- retorna-o como resposta

A função input pode receber no máximo um argumento que deve ser uma **string** e será exibida ao usuário.

Função input

Exemplo: Use o editor para rodar este trecho de código. O que acontece?

```
nome = input('Seu nome: ')
valor = input('Valor do empréstimo: ')
parcelas = input('Quantidade de parcelas: ')
print(nome, ', a dívida será de:', valor *
parcelas)
```

Função input

Corrigindo com os tipos de variáveis atribuídos corretamente.

```
nome = input('Seu nome: ')
valor = float(input('Valor do empréstimo: '))
parcelas = int(input('Quantidade de parcelas: '))
print(nome, ', a dívida será de:', valor * parcelas)
```

Função input

Entradas na mesma linha em Python.

```
a, b, c = input().split()
```

Depois dizer o tipo de variáveis de a, b e c, ou seja:
a = int(a)
b = float(b)
c = float (c)

```
a, b, c = map(int, input().split())
```

Função input

```
entrada = input("Digite real, inteiro, booleano e string: ").split()

real = float(entrada[0])
inteiro = int(entrada[1])
booleano = entrada[2] == "True"   # cuidado: bool("False") também dá True!
string = entrada[3]

print(real, inteiro, booleano, string)
```

Função input

Como o python interpreta esses comandos?

1) Inicialmente `input` será executada, pois é a função mais interna:
   `valor = float(input('Valor do empréstimo: '))`
2) Supondo que o usuário dê a entrada `1000.00`, a função `input` converterá esse valor para *string* e a retornará como resposta:
   `valor = float('1000.00')`
3) Agora a função `float` converterá seu argumento, que neste caso é uma *string*, para o correspondente número real e retornará esse valor.
   `valor = float('1000.00')` ⇨ `valor = 1000.0`
4) Por fim, o valor `1000.0`, de fato um número real representado em ponto flutuante, será atribuído à variável.
   `valor = 1000.0`

Tipos de erros comuns

1. **Erros de Sintaxe**: o nome é autoexplicativo; em geral são fáceis de se corrigir. O interpretador não entende o que deve fazer por conta desses erros.
2. **Erros em tempo execução:** ocorrem só quando o programa é executado - pois não há erro de sintaxe - devido a uma instrução que não pode ser cumprida, uma violação do uso esperado do problema, etc. São chamados de **exceções.** O interpretador entende o que deve fazer mas não consegue executar a tarefa.
3. **Erros de lógica / semântica:** São erros relacionados ao algoritmo. São os mais difíceis de resolver. O interpretador entende o que deve fazer, executa a tarefa mas isso ainda não resolve o problema.

FIM